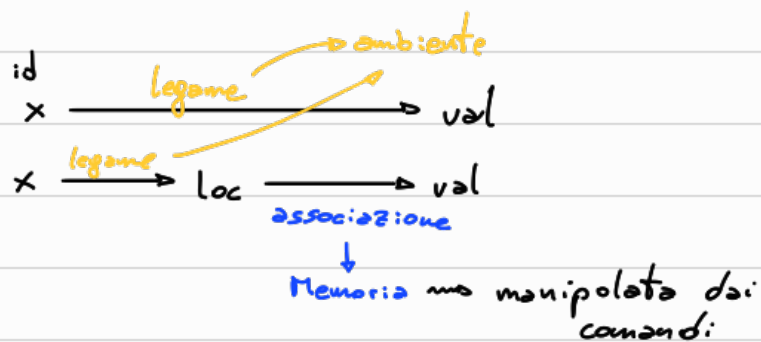


Ambienti vs memorie



no "integrazione" tra memorie e ambiente

come creiamo l'ambiente in cui vogliamo eseguire del codice

Blocco no comandi

↳ porzione di programma (potenzialmente isolata da altri pezzi di codice)

Blocchi:

- Permettono di rendere modulare il codice e quindi di renderlo più chiaro
 - Permettono la gestione locale di identificatori
 - Permettono di gestire l'allocazione della memoria
 - Permettono la ricorsione
- ↳ maggiormente legati ai blocchi come procedure

D; C
↓
Blocco inline (senza nome)
specifico le dichiarazioni D
che definiscono l'ambiente
in cui eseguire la porzione di
codice C.

{ blocco } ← alcuni linguaggi possono richiedere delimitatori del blocco

In Imp $C \rightarrow \dots | D; C$ $\Rightarrow$ è possibile inserire dichiarazioni in qualunque punto del programma

no con questa sintassi (e con la semantica che specificheremo) una dichiarazione è visibile da dove viene inserito fino alla fine del programma

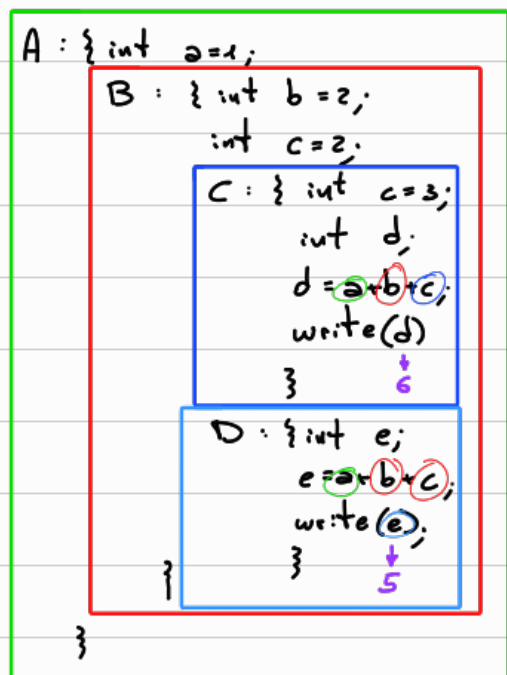
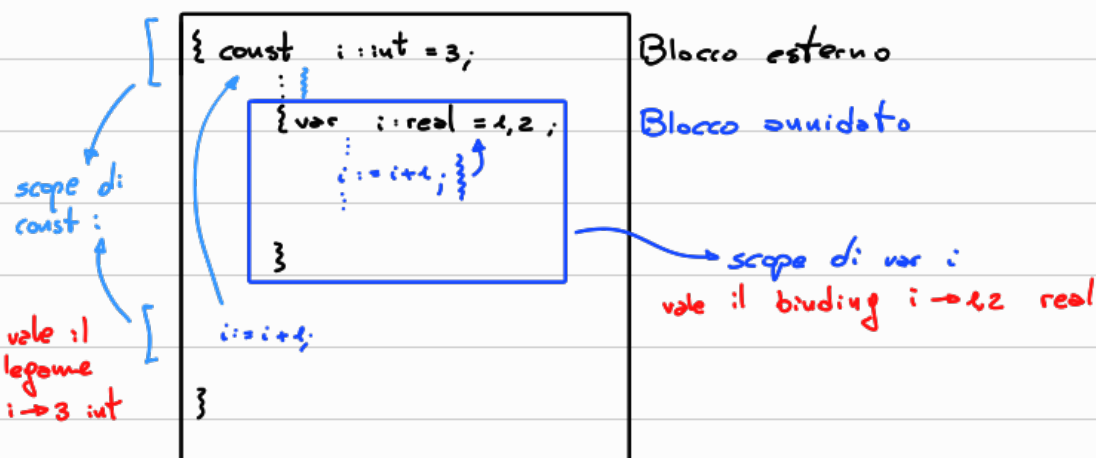
D
↓
visibilità

Blocchi annidati



=> I blocchi possono solo essere contenuti interamente in altri blocchi: ma blocchi annidati
 => Questa regola di sovrapposizione tra blocchi permette di definire la regola di visibilità delle dichiarazioni

Regola di visibilità: Una dichiarazione locale ad un blocco è visibile nel blocco e in tutti i blocchi annidati, a meno che in tali blocchi non intervenga una dichiarazione dello stesso nome



Globale $\rightarrow$ visibile a tutti i blocchi essendo tutti i blocchi annidati in A (unica visibile in A)

Locali $\rightarrow$ B ma vede le sue dichiarazioni locali e quelle di A

Locali $\rightarrow$ C ma c viene ridefinita (nonostante dentro C la precedente dichiarazione di c)

Locali $\rightarrow$ D ma non vede c perché C NON contiene D. D non può vedere l'ambiente locale a C

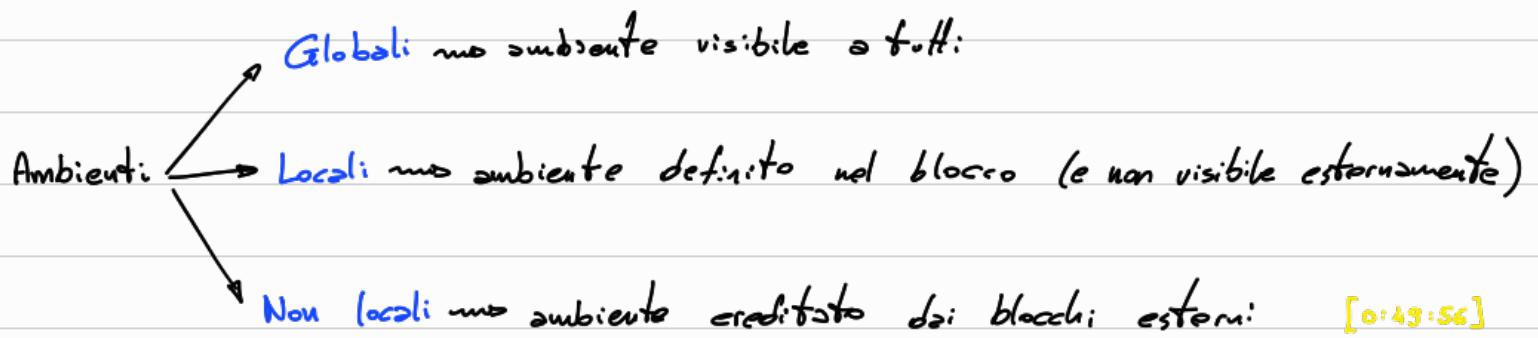
Scope $\rightarrow$ porzione di codice in cui tutte le occorrenze di x fanno riferimento alla dichiarazione di x
 $\hookrightarrow$ devono usare il legame definito dalla dichiarazione

var x :int = 3

$\left\{ \begin{array}{l} \text{regola di visibilit  definisce} \\ \text{lo scope ovvero la posizione di} \\ \text{codice in cui le occorrenze della} \\ \text{variabile dichiarata fanno riferimento} \\ \text{alla dichiarazione} \end{array} \right.$

[0:42:00]

$\rightarrow$ concetto statico che lega un identificatore all'ambiente di riferimento a tempo di compilazione in quanto (senza l'uso di procedura) eseguiamo il codice dove viene definito



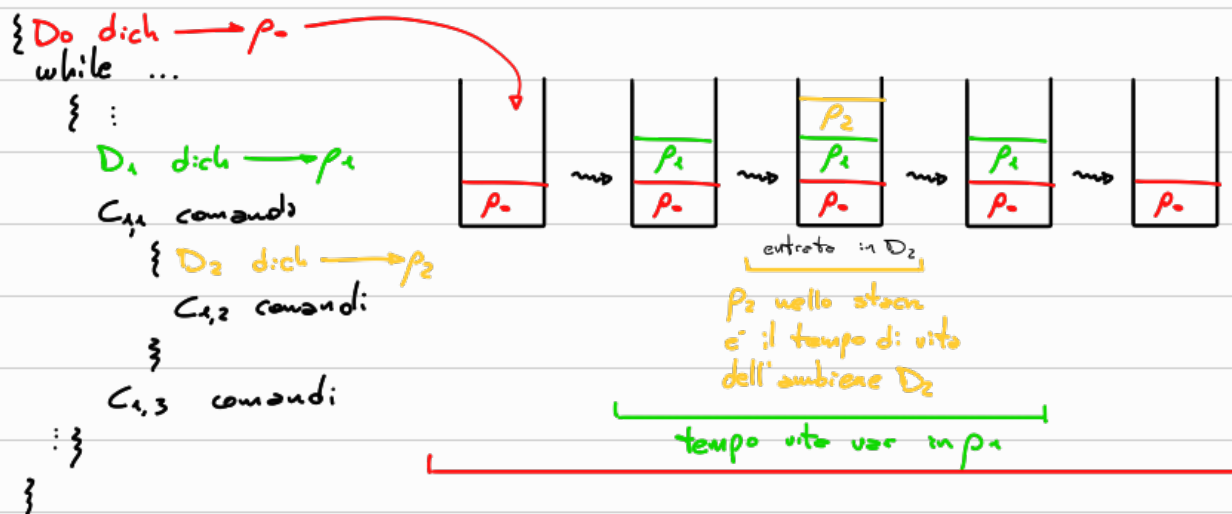
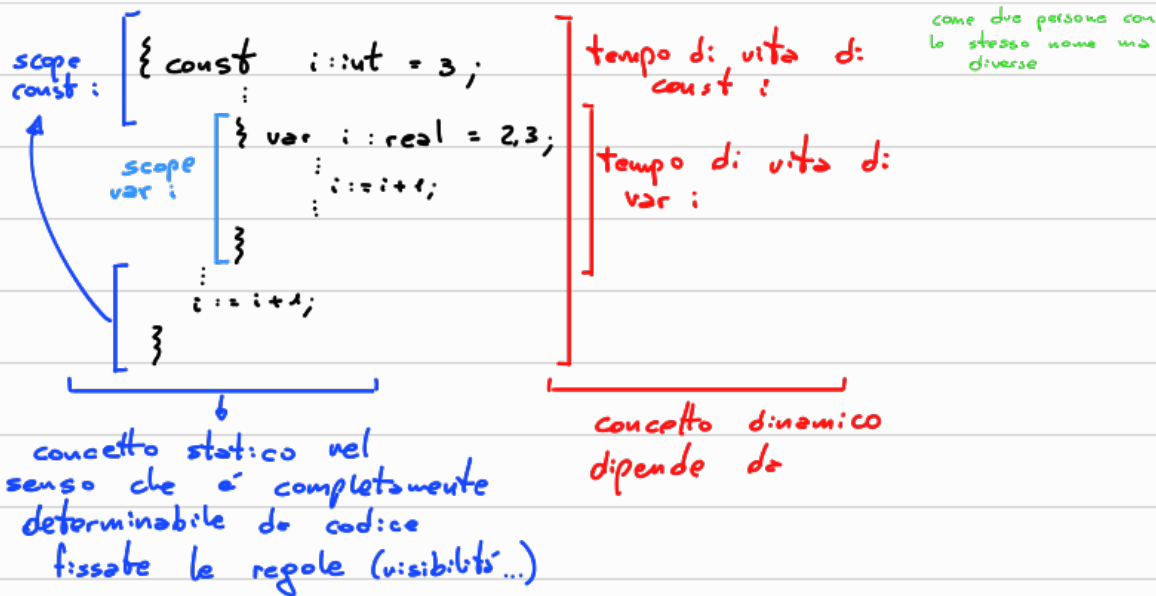
[0:49:56]

SCOPING $\rightarrow$ concetto che riguarda porzioni di codice

$\Rightarrow$ determina e quali legami (ambiente statico) fa riferimento ogni occorrenza. legame di riferimento nel codice

Tempo di vita $\rightarrow$ ogni variabile ha un tempo di vita ovvero il tempo di esecuzione in cui l'associazione della locazione (memoria) rimane valida

$\left[\begin{array}{l} \text{Allocazione in memoria} \\ \text{Deallocazione} \end{array} \right.$



Allocazioni di p_1 e p_2 successive nello stack durante l'esecuzione del while sono di fatto ambienti nuovi, diversi:

La sono stati distrutti e riallocati ad ogni iterazione

Blocchi di imp

con $C \rightarrow x := E \mid \text{skip} \mid \text{while } E \text{ do } C \mid \text{if } E \text{ then } C \text{ else } C \mid \boxed{D; C}$

blocco

semantica statica $\rightarrow$ determina se il comando è ben formato

$$\frac{\Delta \vdash d: \Delta_0 \quad \Delta[\Delta_0] \vdash c}{\Delta \vdash d; c}$$

posizione libera

$$FI(d; c) = FI(d) \cup (FI(c) \setminus DI(d))$$

$$DI(d; c) = DI(d) \cup DI(c)$$

Semantica dinamica

$$\frac{\rho \vdash \langle d, \sigma \rangle \rightarrow^* \langle \rho_0, \sigma' \rangle}{\rho \vdash \langle d; c, \sigma \rangle \rightarrow \langle \rho_0; c, \sigma' \rangle}$$

$$\frac{\rho[\rho_0] \vdash \langle c, \sigma' \rangle \rightarrow^* \sigma'}{\rho \vdash \langle \rho_0; c, \sigma' \rangle \rightarrow \sigma'}$$

? ↓
↑ ?

[1:24:00]

Esempio

var x: int = 3; x := x + 1

Sem. statica

$$\frac{? \vdash d: \Delta \quad \Delta \vdash c}{\vdash d; c} \text{ Regola } *$$

$$\frac{\vdash 3: \text{int}}{\vdash \text{var } x: \text{int} = 3: [x \mapsto \text{intloc}] \equiv \Delta}$$

Risolviamo la regola *

$$\frac{\vdash d: [x \mapsto \text{intloc}] \quad [x \mapsto \text{intloc}] \vdash c \text{ ? } \checkmark}{\vdash d; c}$$

$$\frac{[x \mapsto \text{intloc}] \vdash x + 1: ?}{[x \mapsto \text{intloc}] \vdash x: x + 1} \Delta(x) = \text{int}$$

$$\frac{[x \mapsto \text{intloc}] \vdash x: \text{int} \quad [x \mapsto \text{intloc}] \vdash 1: \text{int}}{[x \mapsto \text{intloc}] \vdash x + 1: \text{int}} \Delta(x) = \text{int}$$

- => espressione ben formata
- => assegnamento ben formato
- => blocco ben formato

Sem. dinamica

$$\frac{}{\vdash \langle d; c, \emptyset \rangle \rightarrow}$$

↑
mem. iniziale vuota